

COS214 Practical 4

Team:

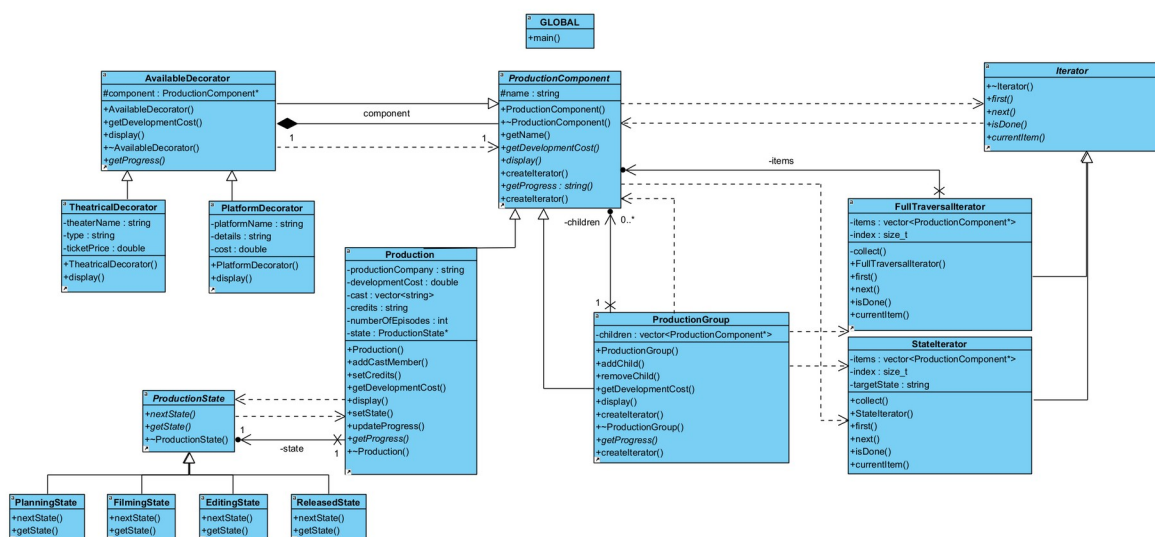
- u24876489 – Katlego Sentsho
- u23545756 – Kyle Pillay
- u25108582 – Jonathan Witter

System Description

We decided on film production for the domain. This design allows for Studios to group their films (produced or in progress of being made) into their type of film (Movie, Tv Series, Animated, ...)

Films will have states. They are either in planning/filming/editing phase or they have been released. Films are decorated with how they are accessed (viewed) by the public (such as online platforms for streaming or theaters).

UML Diagram



GoF Participants

Composite Design Pattern:

- Component – ProductionComponent
- Composite – ProductionGroup
- Leaf – Production

State Design Pattern:

- Context – Production
- State – ProductionState
- Concrete States – Planning, Filming, Editing, Released

Decorator Design Pattern:

- Component – ProductionComponent
- Concrete Component – Production
- Decorator – AvailableDecorator
- Concrete Decorators – TheatricalDecorator, PlatformDecorator

Iterator Design Pattern:

- Iterator – Iterator
- Concrete Iterators – FullTraversalIterator, StateIterator
- Aggregate – ProductionComponent
- Concrete Aggregates – ProductionGroup

Design and Ownership Rationale:

ProductionGroup owns its children by raw pointer in a `std::vector<ProductionComponent*>` and deletes each one in its destructor, so destroying a group cleanly cascades through its whole subtree. `removeChild()`

deliberately hands ownership back to the caller rather than deleting internally, which is what lets main.cpp relocate or delete a node explicitly, satisfying the requirement for a meaningful structural change at runtime. Both Production (leaf) and ProductionGroup (composite) inherit from ProductionComponent, which declares getDevelopmentCost(), display(), getProgress(), and createIterator() as shared behaviour, so client code and decorators can treat a single production and an entire group identically without ever checking which one they're holding.

AvailableDecorator also owns what it wraps, holding a ProductionComponent* and deleting it in its own destructor. This means a whole chain of stacked decorators, such as two PlatformDecorators wrapped around one Production, is destroyed correctly just by deleting the outermost layer, and a fully decorated object can be dropped straight into a group's children vector with no special handling needed for cleanup.

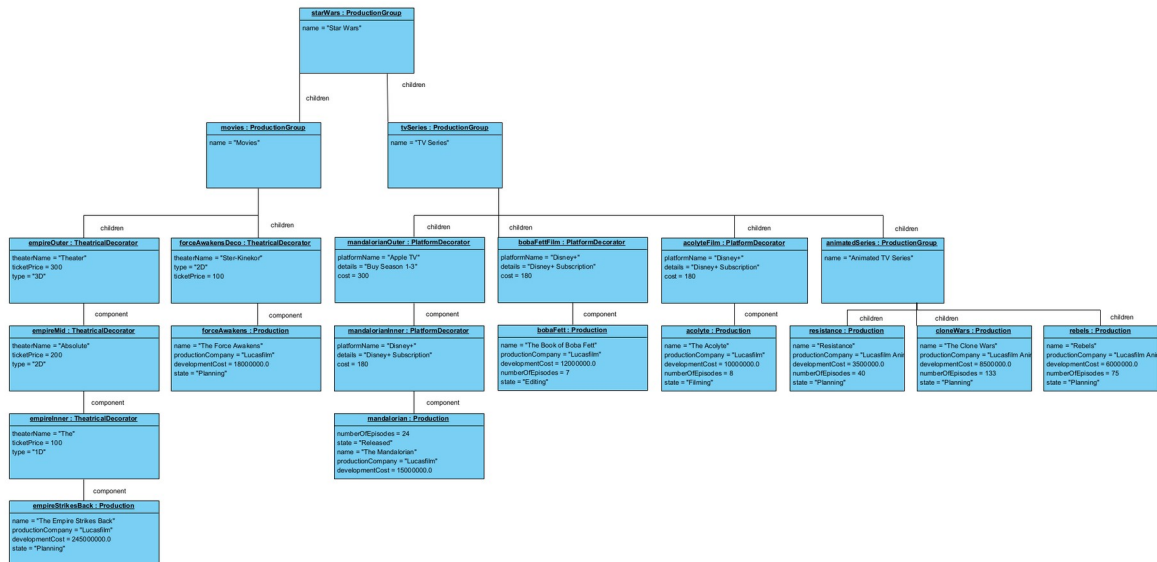
Production owns a single ProductionState*, replacing it through setState(), which deletes the old state before assigning the new one. States are stateless data-wise, encoding only behaviour, so a fresh object is created on each transition rather than reusing shared instances, a simpler choice for a system this size.

FullTraversalIterator and StateIterator both implement the same Iterator interface but serve different purposes, one lists the entire hierarchy and the other filters leaves by lifecycle state. Both are snapshot iterators, collecting their results once at construction through a private collect() method, and both are declared friends of ProductionGroup so they can walk its private children vector directly without that vector ever being exposed to client code.

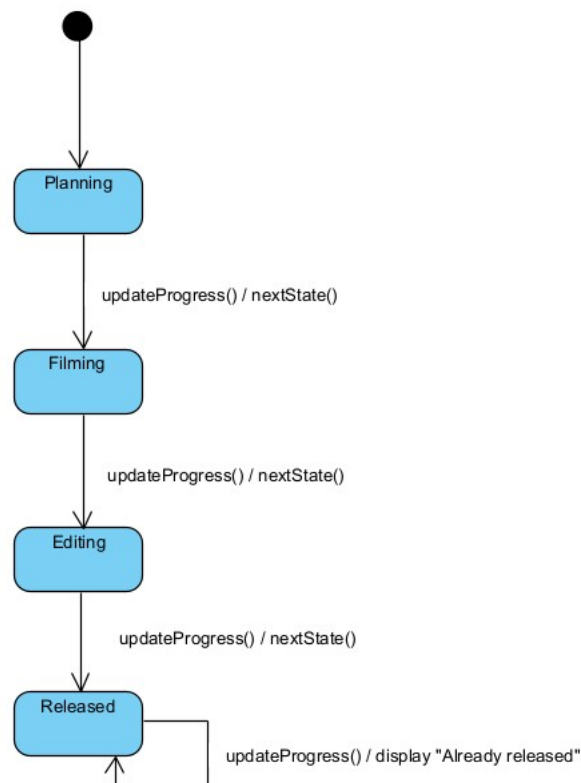
The main design challenge was making these four patterns cooperate rather than function as separate demos. Since a Production can be wrapped in decorators while sitting inside a composite tree, AvailableDecorator delegates every ProductionComponent method to the component it wraps, so decorated and undecorated productions remain fully interchangeable, and traversal, state, and cost calculations all keep working correctly no matter how many decorator layers are present.

Diagrams

Object Diagram:

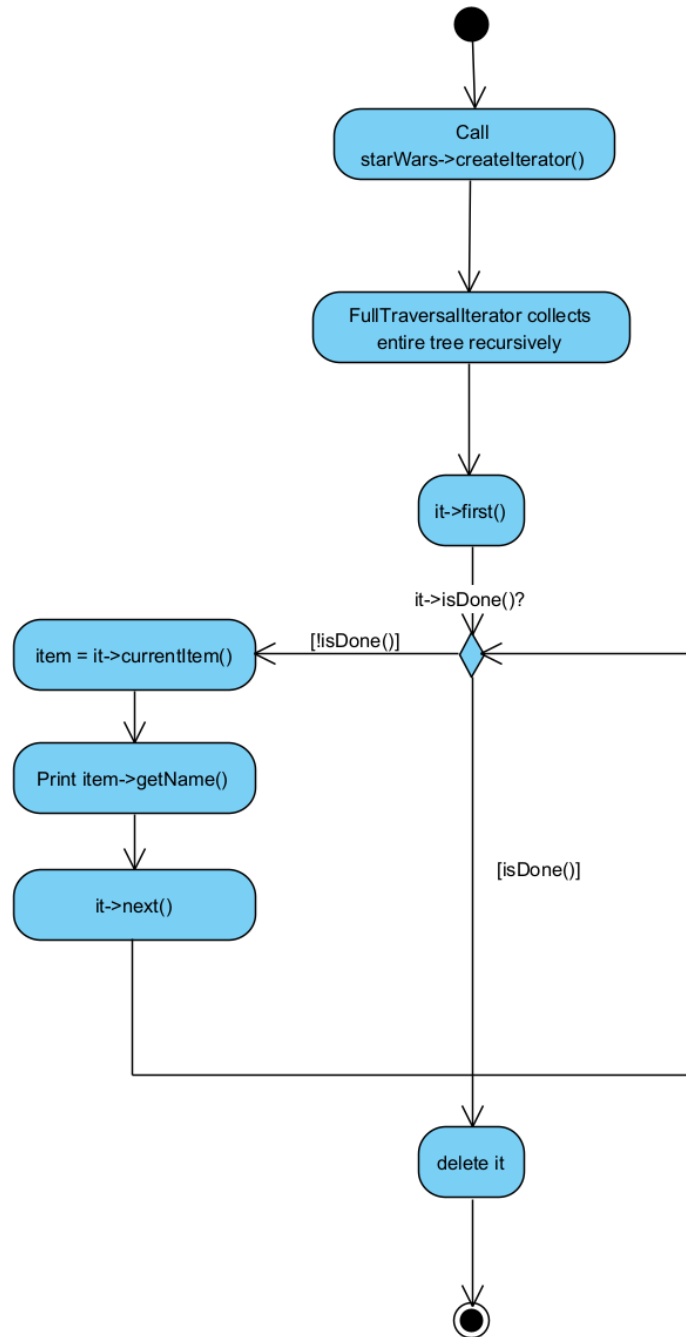


State Diagram:

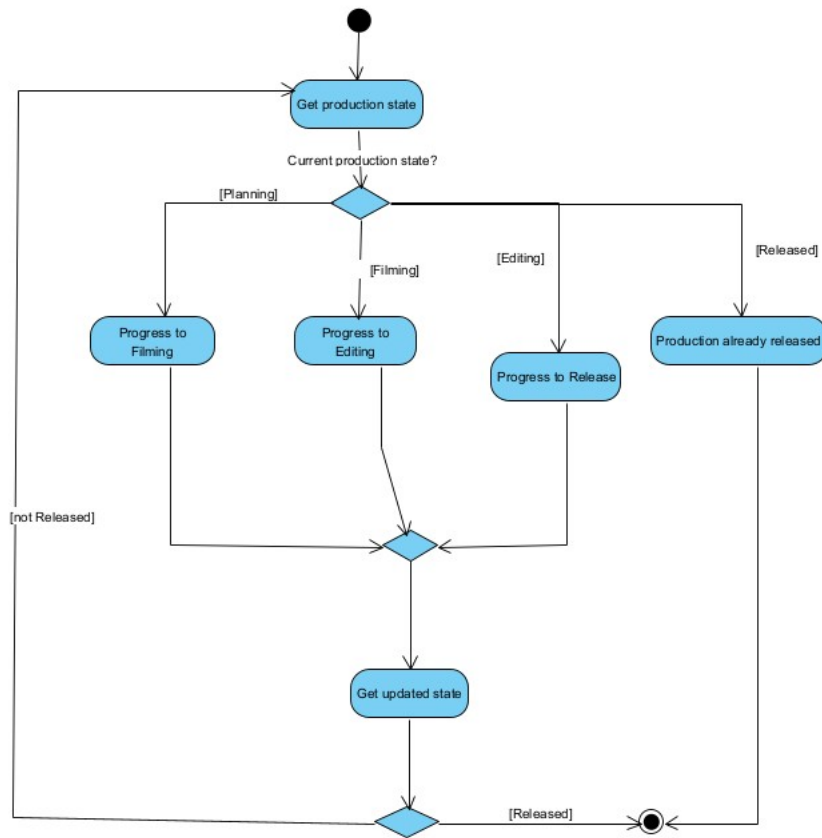


Traversal Activity Diagram:

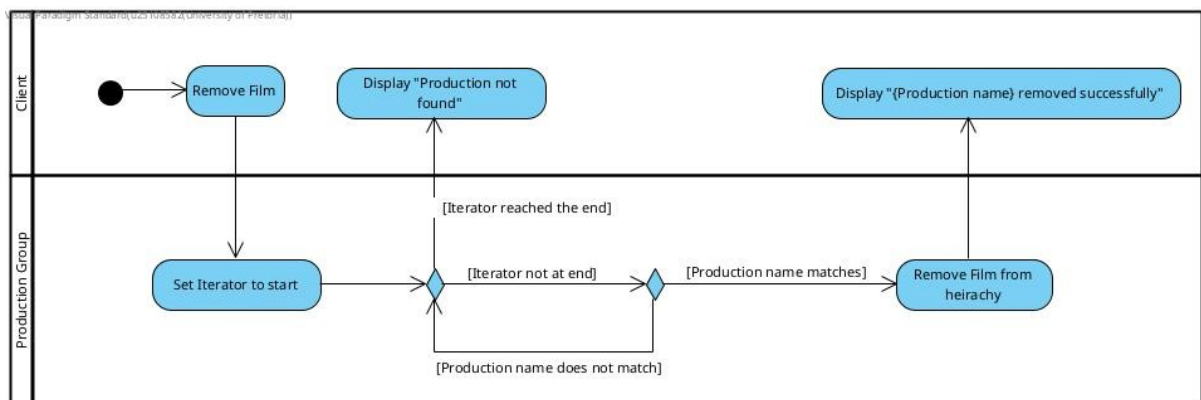
act [Full Hierarchy Traversal]



LifeCycle Activity Diagram:



Remove Production Activity Diagram:



Traversal Modification Policy

Our iterators use a snapshot approach rather than a live one. When `FullTraversalIterator` or `StateIterator` is created, its `collect()` method walks the entire composite tree once, right at construction, and copies pointers to the relevant components into its own internal vector. From that point on, `first()`, `next()`, `isDone()`, and `currentItem()` only ever look at this internal copy, never the live tree again.

This means once an iterator exists, it's frozen in time. If you add a child with `addChild()` or remove one with `removeChild()` after the iterator was created, that change simply won't show up in the iterator's results, it'll keep returning whatever it originally collected.

We chose this deliberately over a live traversal. A live traversal that kept reading directly from `ProductionGroup`'s children vector would be vulnerable to the classic problem of modifying a container while iterating over it, especially risky in our case since `main.cpp` deletes removed nodes immediately after calling `removeChild()`. Snapshotting sidesteps that risk entirely, since the iterator's own vector is separate from the live tree, so deleting a node from the tree can't invalidate an iterator that's already finished collecting.

This approach also lets two independent traversals run over the same hierarchy at once without stepping on each other. In `main.cpp`, we create a full traversal and a state-filtered traversal one after another from the same root, and each one works from its own private snapshot, so neither interferes with the other or with the underlying tree.

GDB and Valgrind

```
Breakpoint 1, ProductionGroup::removeChild (this=0x555555770c0, name="The Force Awakens")
    at /home/chweeks/cos/COS214/Prac4 - Final/ProductionGroup.h:33
33      ProductionComponent *removeChild(const std::string &name)
(gdb) continue
Continuing.

Program received signal SIGSEGV, Segmentation fault.
__memcpy_avx_unaligned_erms () at ../sysdeps/x86_64/multiarch/memmove-vec-unaligned-erms.S:342
warning: 342  ../sysdeps/x86_64/multiarch/memmove-vec-unaligned-erms.S: No such file or directory
(gdb) backtrace
#0  __memcpy_avx_unaligned_erms () at ../sysdeps/x86_64/multiarch/memmove-vec-unaligned-erms.S:342
#1  0x00007ffff7d0b876 in void std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>>::_M_construct<char*>(char*, char*,
std::forward_iterator_tag) () from /lib/x86_64-linux-gnu/libstdc++.so.6
#2  0x0000555555559567 in ProductionComponent::getName[abi:cxx11]() const (this=0x55555577110)
    at /home/chweeks/cos/COS214/Prac4 - Final/ProductionComponent.h:20
#3  0x00005555555585dd in main () at main.cpp:154
```

```

==1043== 73,728 bytes in 1 blocks are still reachable in loss record 49 of 49
==1043==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1043==    by 0x491738E: ??? (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.33)
==1043==    by 0x400571E: call_init.part.0 (dl-init.c:74)
==1043==    by 0x4005823: call_init (dl-init.c:120)
==1043==    by 0x4005823: _dl_init (dl-init.c:121)
==1043==    by 0x401F59F: ??? (in /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2)
==1043==
==1043== LEAK SUMMARY:
==1043==    definitely lost: 0 bytes in 0 blocks
==1043==    indirectly lost: 0 bytes in 0 blocks
==1043==    possibly lost: 0 bytes in 0 blocks
==1043==    still reachable: 77,666 bytes in 49 blocks
==1043==    suppressed: 0 bytes in 0 blocks
==1043==
==1043== For lists of detected and suppressed errors, rerun with: -s
==1043== ERROR SUMMARY: 7 errors from 6 contexts (suppressed: 0 from 0)
make: *** [makefile:12: memory] Segmentation fault (core dumped)

```

Problem: the program crashes with a segmentation fault shortly after removing "The Force Awakens" from the movies group, specifically when trying to print the removed production's name.

Cause: `ProductionGroup::removeChild()` both erased the child from the children vector and called `delete removedChild;` on it, before returning the same now-dangling pointer to the caller. `main.cpp` then used that returned pointer, first reading `removed->getName()`, and intended to delete `removed;` afterwards, unaware the object had already been destroyed inside `removeChild()`. This violated the intended ownership contract, `removeChild()` was meant to only detach the child and transfer ownership to the caller, never destroy it itself.

Final Valgrind evidence:

```

==22==
==22== HEAP SUMMARY:
==22==    in use at exit: 0 bytes in 0 blocks
==22==    total heap usage: 100 allocs, 100 frees, 77,983 bytes allocated
==22==
==22== All heap blocks were freed -- no leaks are possible
==22==
==22== For lists of detected and suppressed errors, rerun with: -s
==22== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

GitHub

<https://github.com/JM-Witter/COS214---Prac-4.git>

Contributions

Each team member participated in group meeting to discuss the practical and overall design.

Each member contributed at least one of the key classes (composite, iterator, decorator, etc) and fully implemented them.

Each member also completed at least 2 of the 6 diagrams required.

- Katlego Sentsho: Composite, State Diagram, Activity Diagram, PDF work
- Kyle Pillay: Iterator, Object Diagram, State Diagram, PDF work
- Jonathan Witter: State, Decorator, UML Diagram, Activity Diagram, Docker, PDF work

Evidence:

main	All users	All time
Commits on Sep 7, 2026		
Updated UML to have StateIterator+Fixes	Verified KP23545756 authored 12 minutes ago	576297f
Completed Activity Diagram for Traversal	Verified KP23545756 authored 1 hour ago	1616711
Object Diagram completed	Verified KP23545756 authored 1 hour ago	194e0ae
Merging Iterator to Main	JM-Witter committed 3 hours ago	2ffc030
Finished of the iterators	Verified KP23545756 authored 3 hours ago	d36a6e1
Added state's iterator	Verified KP23545756 authored 3 hours ago	861e275
Added PDF	JM-Witter committed 4 hours ago	45d23f9
Updated makefile	JM-Witter committed 4 hours ago	94e7ef3
Added docs folder Moved designs to docs folder	JM-Witter committed 4 hours ago	9333675
Updated ReadMe	JM-Witter committed 4 hours ago	e005cfb
Added docker	JM-Witter committed 4 hours ago	132810b
Added UML.vpp file	JM-Witter committed 4 hours ago	a705c2a
Added UML (WIP)	JM-Witter committed 4 hours ago	2e34cc1
Merging decorator to main	JM-Witter committed 4 hours ago	0fb0512
Added State to main Changed Production display to add state	JM-Witter committed 5 hours ago	1b5af21

iterator	All users	All time
Commits on Sep 7, 2026		
Finished of the iterators	Verified KP23545756 authored 3 hours ago	d36a6e1
Added state's iterator	Verified KP23545756 authored 3 hours ago	861e275
Placeholder for future decorator traversal	Verified KP23545756 authored 9 hours ago	6833b32
Full Traversal iterator with accompanying main	Verified KP23545756 authored 9 hours ago	c01b1d5
Added changes for new iterator class	Verified KP23545756 authored 9 hours ago	6b1e663

composite

All usersAll time

Commits on Sep 6, 2026

composite

kat2005-sen committed yesterday

74ff516

Commits on Sep 2, 2026

Added MakeFile | Added Design Ideas

JM-Witter committed 5 days ago

81fd9bbb

Initial commit

JM-Witter authored 5 days ago

Verified1d60d57

main

6 Branches0 Tags

Switch branches/tags

Find or create a branch...

Branches

Tags

✓ main

UML

composite

decorator

iterator

state

View all branches